IE 7615 Discriminative Deep Learning Project

Milestone 1: Single-Object Image Identification with CNNs

Group 2

Members :

Yi-Fan Liu
Pooja Yogesh Patil
Hiral Hitesh Rana
Wen- Hsin su

1. Dataset and Problem Overview

The goal of this milestone is to identify which object appears in a single-object image. Each student in the class photographed one unique object under different lighting conditions, angles and backgrounds, which gives a 73-class image classification problem (OBJ001 to OBJ073). We used the final merged dataset and the final Object IDs published by the instructors.

Each object has its own folder, and the folder name is the class label. Background-only shots (files marked _bg_) contain no object and are kept for the Week 2 multi-object composites, so they were excluded here. This leaves 6,964 object images, with 90 to 106 images per class (mean 95.4). The classes are close to balanced, so no class weighting was needed. All images were resized to 224 × 224 pixels.

The data was split 70 / 15 / 15 into training (4,874 images), validation (1,045) and test (1,045) sets. The split is stratified by class with a fixed seed (42), so every object appears in all three sets in the same proportions, and every model was trained and evaluated on the same images. The test set has only about 14 to 15 images per class and was used only for final reporting, never for training or model selection.

To reflect the variation in how the photos were taken, the training images were augmented on the fly with random horizontal flips, rotation (±8%), zoom (±15%), translation (±10%), brightness (±20%) and contrast (±20%). Augmentation is active only during training.

2. Models Tested

We compared one CNN built from scratch with four CNNs pretrained on ImageNet (Table 1). All models were implemented in TensorFlow 2.20 / Keras 3 and trained on a Google Colab T4 GPU.

Model	Type	Params (M)	Classification head	Layers unfrozen in Stage 2
Custom CNN	From scratch	1.61	GAP → Dropout 0.4 → Dense	All layers trained
VGG16	Transfer learning	14.75	GAP → Dropout 0.3 → Dense	block5
ResNet50	Transfer learning	23.74	GAP → Dropout 0.3 → Dense	conv5 stage
MobileNetV2	Transfer learning	2.35	GAP → Dropout 0.3 → Dense	From block_13
EfficientNetB0	Transfer learning	4.14	GAP → Dropout 0.3 → Dense	From block6a

Table 1. Models compared. GAP = global average pooling; Dense = 73-way softmax output.

Custom CNN / Simple CNN (baseline). Five convolution blocks (Conv 3×3 → Batch Normalization → ReLU → Max Pooling) with 32, 64, 128, 256 and 512 filters, followed by global average pooling, dropout (0.4) and a softmax layer. It was trained end to end for up to 60 epochs with Adam (learning rate 0.001), early stopping on validation accuracy (patience 10), and a learning-rate schedule that halves the rate when validation loss stops improving for 4 epochs.

Transfer learning models. Each pretrained backbone was used with its original ImageNet preprocessing and trained in two stages. In Stage 1 (feature extraction), the backbone was frozen and only the new head was trained with Adam at a learning rate of 0.001 for up to 15 epochs. In Stage 2 (fine-tuning), the top block(s) of the backbone were unfrozen and trained together with the head at a learning rate of 0.0001 for up to 15 more epochs. Batch Normalization layers stayed frozen throughout, which keeps fine-tuning stable on a small dataset. Both stages used early stopping on validation accuracy (patience 4).

For every model, the checkpoint with the highest validation accuracy across both stages was kept and used for testing. All models used a batch size of 32 and sparse categorical cross-entropy loss.

3. Experimental Results

Table 2 summarizes the results. Models are ranked by validation accuracy, which is the metric we used to choose the final model. Test accuracy, top 5 accuracy and macro-averaged F1 (each class weighted equally) are reported on the held-out test set.

Model	Val acc	Test acc	Top-5	Macro F1	Params (M)	Size (MB)	Epochs	Train time (min)
EfficientNetB0	**98.2%**	**97.4%**	**99.7%**	**0.974**	**4.14**	**16.6**	**30**	**9.9**
ResNet50	97.5%	97.0%	99.6%	0.970	23.74	94.9	28	16.5
MobileNetV2	96.2%	96.2%	99.3%	0.962	2.35	9.4	29	7.1
VGG16	95.5%	95.0%	98.7%	0.950	14.75	59.0	23	13.8
Custom CNN	77.0%	78.5%	92.6%	0.785	1.61	6.4	55	23.2

Table 2. Results for all five models (test set: 1,045 images). The selected model is highlighted.

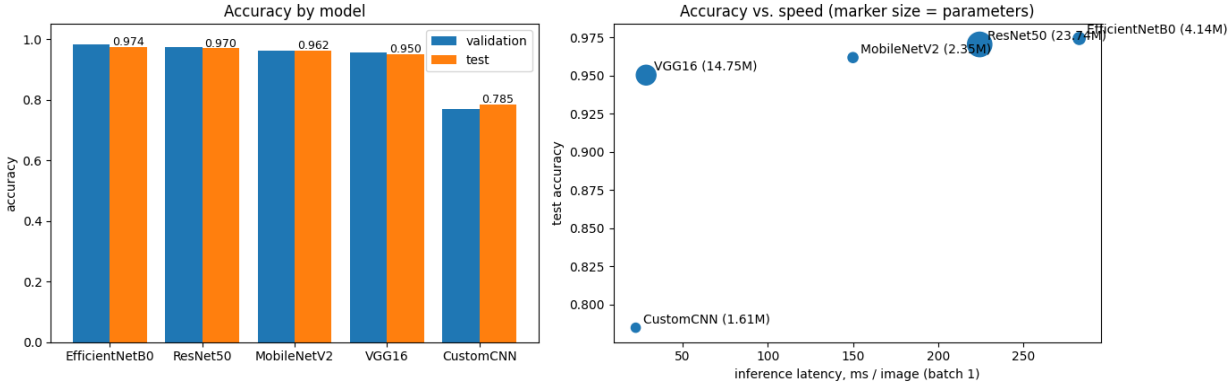


Figure 1. Validation and test accuracy by model (left), and test accuracy against measured inference latency (right).

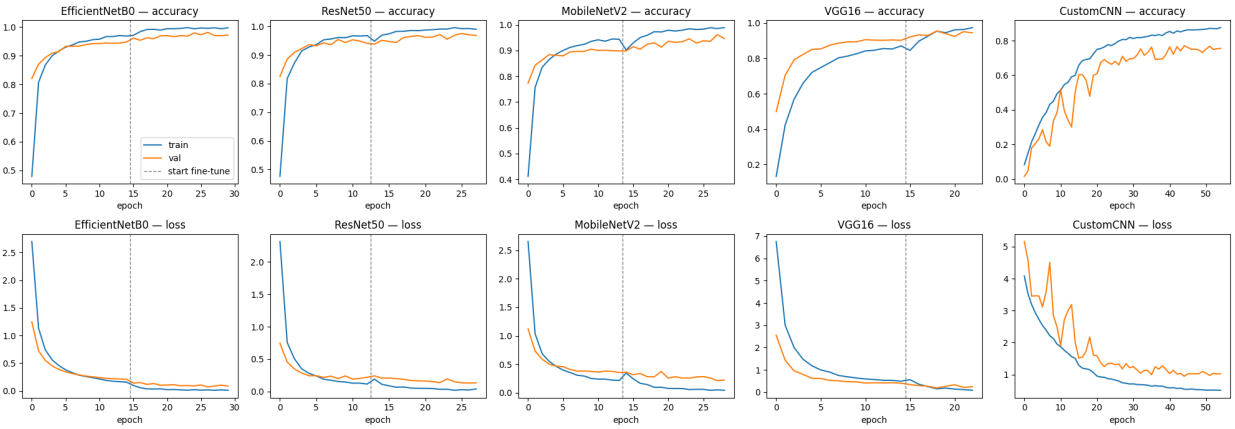


Figure 2. Training and validation curves. The dashed line marks the start of fine-tuning (Stage 2).

Fine-tuning improved every pretrained model. Comparing the best validation accuracy at the end of Stage 1 with the best after Stage 2, MobileNetV2 rose from 90.5% to 96.2% (+5.6 points), VGG16 from 90.6% to 95.5% (+4.9), EfficientNetB0 from 94.8% to 98.2% (+3.4) and ResNet50 from 95.4% to 97.5% (+2.1). The jump is visible right after the dashed line in Figure 2.

4. Comparison and Analysis

4.1 Transfer learning versus training from scratch

All four pretrained models reach 95% to 97.4% test accuracy, while the custom CNN reaches 78.5%, a gap of 16.5 to 18.9 points. The reason is the amount of data. With about 67 training images per object, a network trained from scratch must learn low-level features such as edges and textures as well as the differences between objects. The pretrained backbones already learned general visual features from 1.2 million ImageNet images, so they only need to adapt those features to our 73 objects.

The custom CNN's curves (Figure 2, right) confirm this. Its validation accuracy is very unstable during the first 20 epochs (for example, 51.8% at epoch 11 and 30.1% at epoch 14), and it ends with training accuracy near 87.5% against validation accuracy around 75%, so it also overfits more than the pretrained models. It needed 55 epochs and 23.2 minutes, the longest training time of all five models, and early stopping ended training after validation accuracy stopped improving at epoch 45. Its top 5 accuracy of 92.6% shows it often ranks the correct object among its first few guesses, but it cannot separate similar objects reliably.

4.2 Comparison of the pretrained models

The four pretrained models are within 2.4 points of each other, so efficiency matters when comparing them. EfficientNetB0 is the most accurate and has only 4.14 million parameters. ResNet50 is almost as accurate (97.0%) but is about 5.7 times larger (94.9 MB) and took 1.7 times longer to train. VGG16 is the least accurate pretrained model despite having 14.75 million parameters, which is expected from an older architecture without residual connections or efficient blocks. MobileNetV2 is the smallest (2.35 million parameters, 9.4 MB) and fastest to train (7.1 minutes) while still reaching 96.2%.

All pretrained models overfit only mildly: training accuracy ends near 97% to 99%, while validation accuracy stays between 95% and 98%, and validation loss does not rise again during fine-tuning. Augmentation, dropout and early stopping kept the gap small even though the dataset is small.

4.3 Error analysis of the best model

EfficientNetB0 misclassified 27 of the 1,045 test images (2.6% error). The confusion matrix (Figure 3) has a strong diagonal, and no pair of objects is confused more than twice. The weakest class is OBJ061 (recall 0.80, 12 of 15 correct). OBJ013 has the lowest precision (0.72), because other objects are sometimes predicted as OBJ013. Several errors involve OBJ013, OBJ059 and OBJ061 being mistaken for one another (OBJ061 → OBJ013 twice, OBJ013 → OBJ059, OBJ059 → OBJ061), which suggests these objects share visual features. OBJ034 was predicted as OBJ062 twice.

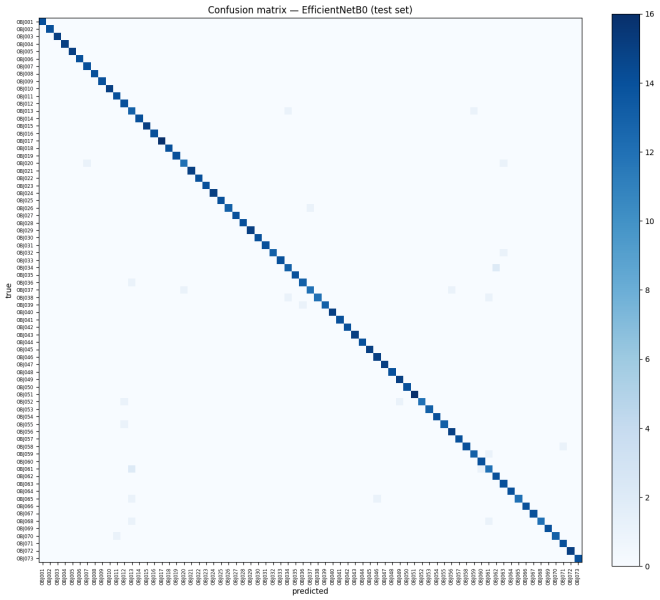


Figure 3. Confusion matrix of EfficientNetB0 on the test set.



Figure 4. Examples of misclassified test images (true label, predicted label and confidence).

Figure 4 shows why these errors happen. In most of them the object is very small in the frame, partly hidden, in a dark scene, or surrounded by clutter, so the background dominates the image. Most wrong predictions also have low confidence (many between 0.33 and 0.63), which means a confidence threshold could flag uncertain predictions in a real attendance system instead of returning a wrong ID.

5. Final Model Selection and Justification

We selected EfficientNetB0 with two-stage fine-tuning as the final model. It has the highest validation accuracy (98.2%), which was the selection criterion, and it also has the highest test accuracy (97.4%), macro F1 (0.974) and top 5 accuracy (99.7%). Because the model was chosen on validation accuracy, the test result is an unbiased estimate of its performance. It achieves this with a small model (4.14 million parameters, 16.6 MB) and under 10 minutes of training. ResNet50 offers no accuracy advantage for its much larger size, and VGG16 is both larger and less accurate. MobileNetV2 is a good alternative if model size or speed is the main constraint, at a cost of about 1.2 points of test accuracy.

Limitations

- The dataset is small (about 95 images per object), and the test set has only 14 to 15 images per class, so one error changes a class's recall by about 7 points. Per-class results should be read with this in mind.
- Each model was trained once with one seed. Repeated runs would show how much the ranking of the closely matched pretrained models depends on random variation.
- Latency was measured with single-image calls, which mostly reflects the number of layers rather than computation. This is why VGG16 measured faster than MobileNetV2. Batched or optimized inference should be measured before using latency to choose a model for deployment.
- Some images show the object very small, partly hidden or in poor light, and these account for most of the remaining errors.

Next steps

Milestone 2 moves to multi-object images, where each object must be identified and located. The main lesson from this milestone, that transfer learning clearly outperforms training from scratch on a dataset this size, supports fine-tuning a pretrained YOLOv8 detector. We will keep the same 73 Object IDs and class order and use the background images set aside here to build the multi-object composites.

GitHub: https://github.com/yvonneliuYL/IE7615-Group2-Discriminative-Project